GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints

Ainslie, Lee-Thorp, de Jong, Zemlyanskiy, Lebrón and Sanghai (EMNLP 2023), arXiv:2305.13245

Paper summary for the CS-781 capstone

The paper at a glance

Citation

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, S. Sanghai. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP 2023. Google Research.

One-sentence summary

Let *groups* of query heads share one key/value head. Quality stays close to full multi-head attention, while inference is almost as fast as single-KV-head multi-query attention.

Why it matters for our capstone

Our A4 uses GQA: 12 query heads share 4 key/value heads, so its KV-cache is 3× smaller. That matters for batched generation of 128-word continuations.

Where it appears in our slides

Main deck: “A4: efficient decoder”. Glossary: “Grouped-query attention”.

Roadmap

1. The problem
2. The basic idea
3. Experiments
4. Assessment and outlook

Why is generation slow? Memory bandwidth

When a decoder generates one new token, it must **read from memory**:

- all model weights, and
- the cached keys and values of *every previous token*, in every layer and head.

The arithmetic for one token is small, so the GPU mostly waits for data: generation is **memory-bandwidth bound**.

The KV-cache grows with sequence length, batch size, layers and number of KV heads. For long inputs it dominates.

Analogy

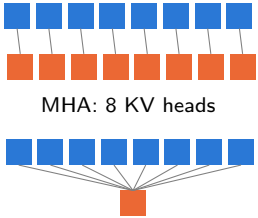
A judge writing an order, one word at a time, must re-read every note taken so far before each word. The thinking is quick; fetching the notes is slow. Fewer, more compact notes means faster writing.

Multi-head vs multi-query attention

- **Multi-head attention (MHA)**: H query heads, H key heads, H value heads. Best quality; largest KV-cache.
- **Multi-query attention (MQA)** (Shazeer, 2019): H query heads share *one* key head and *one* value head. KV-cache is $H\times$ smaller and much faster.

Problems with MQA:

- **quality degradation** and training instability;
- training a separate MQA model just for fast inference is costly;
- popular models (T5, LLaMA 1) were trained with MHA.



Blue = query heads, orange = key/value heads.

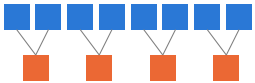
Grouped-query attention (GQA)

Split the H query heads into G **groups**. Each group shares one key head and one value head.

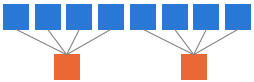
- GQA-1 = MQA (one group).
- GQA-H = MHA (one head per group).
- GQA-G in between: KV-cache is H/G times smaller than MHA.

Why it scales well: larger models have more heads, so MQA's single KV head becomes an ever more *aggressive* cut. GQA keeps the *same proportional* saving as models grow.

GQA is applied to decoder self-attention and cross-attention, not to encoder self-attention, which is computed in parallel.



GQA-4: 4 KV heads, 2 queries per group



GQA-2: 2 KV heads, 4 queries per group

Analogy

MHA: every student has a textbook. MQA: the class shares one. GQA: each table of three shares one.

Easy worked example: our A4's KV-cache

A4: 10 layers, 12 query heads, head size 64.
KV-cache numbers *per token per layer* =
$2 \times$ (KV heads) $\times$ 64.

	KV heads	per token/layer	1,024 tok (fp16)
MHA	12	1,536	31.5 MB
GQA-4 (A4)	4	512	**10.5 MB**
MQA	1	128	2.6 MB

e.g. $1{,}536 \times 1{,}024 \times 10$ layers $\times$ 2 bytes $= 31.5$ MB per sequence.

In our generation script

With a batch of 16 prompts: MHA needs about **503 MB** of KV-cache, GQA-4 about **168 MB**. On a 16 GB T4 this leaves room for bigger batches, so all test prompts generate faster.

A4 also uses sliding windows

Local layers keep only 255 cached keys, so their cache shrinks further. GQA and sliding windows multiply their savings.

Uptraining: converting an existing MHA model

Instead of training a GQA model from scratch:

1. **Convert** the checkpoint: build each group's key (and value) projection by **mean-pooling** the projections of the heads in that group.
2. **Uptrain**: continue pre-training for a small fraction α of the original steps, with the original recipe.

With $\alpha = 0.05$ (5% of the original compute), uptraining T5-XXL took about 600 TPUv3 chip-days.

Mean-pooling example

12 heads into 4 groups of 3. For group 1:

$$W_K^{(g_1)} = \frac{1}{3}\left(W_K^{(1)} + W_K^{(2)} + W_K^{(3)}\right)$$

and the same for W_V. Query projections are unchanged.

This keeps the *average* of what the three heads were looking for, so less pre-trained knowledge is lost than by choosing one head.

Setup

- **Models**: T5.1.1 encoder–decoder. MHA T5-Large and T5-XXL; MQA-XXL and GQA-8-XXL uptrained with $\alpha = 0.05$ from public checkpoints.
- **Tasks**: summarisation (CNN/Daily Mail, arXiv, PubMed, MediaSum, Multi-News), translation (WMT14 EN–DE), question answering (TriviaQA).
- **Fine-tuning**: learning rate 0.001, batch 128, dropout 0.1; inputs up to 2,048 tokens and outputs up to 512 for long summarisation; greedy decoding.
- **Timing**: time per sample on TPUv4, with each model's parallelisation optimised separately.

Why no GLUE?

Classification tasks do not generate text autoregressively, so the KV-cache bottleneck does not arise. The paper focuses on generation-heavy tasks, as are our Phases 1 and 6.

Main results

Model	Time/sample	Average	CNN R1	PubMed R1	MultiNews R1	TriviaQA F1
MHA-Large	0.37	46.0	42.9	46.2	46.6	78.2
MHA-XXL	1.51	**47.2**	**43.8**	47.5	46.9	**81.9**
MQA-XXL	**0.24**	46.6	43.0	46.9	46.5	81.3
GQA-8-XXL	0.28	47.1	43.5	**47.7**	**47.2**	81.6

Averages also include arXiv, MediaSum and WMT. Time per sample on TPUv4.

Reading it

GQA-8-XXL is **5.4**× **faster** than MHA-XXL (0.28 vs 1.51) and only 0.1 lower on average. It is nearly as fast as MQA (0.28 vs 0.24) but 0.5 better. Even MQA-XXL beats MHA-Large on both speed and quality.

Ablations

How to convert

On T5-Large uptrained to MQA: **mean-pooling** beat keeping the first head, which beat random initialisation.

The more pre-trained information is preserved, the better.

How much uptraining

GQA is already reasonable right after conversion; MQA needs uptraining to be useful.

Both gain from 5% uptraining, with diminishing returns at 10%.

How many groups

Going from 1 group (MQA) to 8 adds only modest inference time. Cost grows faster as groups approach MHA.

8 groups were chosen as a good middle ground.

GQA in our capstone

- A4 trains GQA **from scratch**, with no uptraining, since the rules require training from scratch.
- Our `Attention` module stores keys and values in the cache *before* repeating them across query heads, so the cache holds only 4 heads.
- The paper evaluated only encoder–decoder models; it predicts that GQA's advantage over MQA should be *stronger* in decoder-only models like A4.

Experiments worth running

- A4 with $n_{kv} \in \{1, 4, 12\}$ (MQA, GQA-4, MHA): validation loss vs generation speed and peak memory.
- Keep parameters matched: fewer KV heads free parameters that can go into d_{ff} or depth.
- Measure tokens per second when generating the test set.

Strengths and limitations

Strengths

- Simple generalisation that contains MHA and MQA as special cases.
- Near-MHA quality at near-MQA speed.
- Cheap conversion of existing checkpoints (5% compute).
- Scales well: the saving stays proportional as models grow.
- Removes wasted replication of KV heads when sharding big models.

Limitations (several stated by the authors)

- Only encoder–decoder models were evaluated.
- No comparison with a GQA model trained from scratch.
- Long-output quality is measured with ROUGE, which the authors call flawed.
- Fewer KV heads may still reduce capacity for some tasks.
- Speed-ups are measured on TPUs; GPU gains depend on kernels.

Future research directions

Suggested by the paper's limitations

- Decoder-only models, where GQA is expected to help more.
- Uptraining vs training from scratch, at equal compute.
- Better evaluation of long generated outputs.

What followed

- GQA became standard in open models, e.g. Llama 2 70B, Mistral 7B and Gemma 2.
- Other KV-cache reductions: multi-head latent attention (DeepSeek-V2, 2024) compresses keys and values into a small latent; cross-layer KV sharing; KV-cache quantisation.

Open questions

- How many KV heads does a model *really* need, per layer?
- Should different layers use different group counts?
- Does sharing KV heads hurt retrieval of rare facts from long contexts?
- For legal NLP: long judgments make the KV-cache the main memory cost. Can GQA plus sliding windows let a free GPU process whole judgments?

Summary

- Decoding is limited by **memory bandwidth**: reading the KV-cache at every step.
- MQA (one KV head) is fast but loses quality; MHA is accurate but slow.
- **GQA**: G groups of query heads, one KV head per group. KV-cache is H/G times smaller.
- Existing MHA models can be converted by **mean-pooling** KV heads and **uptraining** with 5% of the compute.
- GQA-8-XXL: 47.1 average vs 47.2 for MHA-XXL, at 5.4× the speed and close to MQA's speed.
- For us: A4's 4 KV heads cut its KV-cache 3× (31.5 MB to 10.5 MB per 1,024-token sequence).

Reference

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of EMNLP 2023*. arXiv:2305.13245.

Other work cited: Shazeer (2019) Fast Transformer Decoding: One Write-Head is All You Need; Komatsuzaki et al. (2022) Sparse Upcycling; Raffel et al. (2020) T5; Touvron et al. (2023) Llama 2; Jiang et al. (2023) Mistral 7B; DeepSeek-AI (2024) DeepSeek-V2.